Scalable Parallel-in-Time Integration for Acoustics with Time-Varying Wave Speed

Nathan Chapman[1], Andy Piacsek[2]
[1]Department of Computer Science, Central Washington University
[2]Department of Physics, Central Washington University

January 5, 2025

INTRODUCTION

- Equations of motion
- There needs to be computational support for modeling acoustic metamaterials
- Big problems demand high-performance solvers

THE EQUATIONS OF MOTION AND CAUSALITY

The equations of motion (EoM) are at the heart of physics:

$$\sum_i \vec{F}_i = m\vec{a}.$$

- Numerical methods for solving these equations have historically been sequential to follow causality.
- This sequential nature has limited their ability to benefit from parallelism.
- But what if they could?

ACOUSTICS IN EXPANDING VOLUMES

Wave $\psi(t, \vec{x})$ is defined by its density n and phase θ as:

$$\psi(t, \vec{x}) = \sqrt{n_0 + \Delta n(t, \vec{x})} e^{i(\theta_0 + \Delta\theta(t, \vec{x}))}.$$

There are 3 traditional ways to parallelize the solution of a computational problem: CPU parallelization, GPU parallelization, and Distributed computing. While CPU parallelization is more straightforward to implement, GPU parallelization can allow for runtimes to decrease by many orders of magnitudes. Even lower run times can be achieved by combining either of these parallization schemes with running them on multiple machines. This investigation focuses on parallelizing the solution of equations of motion using GPUs and multiple machines.

These approaches can offer massive increases in performance, but only for problems that are well-posed to be parallelized. Traditionally, initial value problems have been unable to be parallelized due their dependance on causality. Several methods have been created to overcome this limitation. These methods include the Parareal algorithm, Multigrid Reduction in Time (MGRIT), Parallel Full Approximtaion Scheme in Space and Time (PFASST). This investigation focuses on the Parareal algorithm.

A. THE PARAREAL ALGORITHM

The Parareal algorithm does not solve a problem directly, but deconstructs the problem in to subproblems that can be solved with traditional methods in parallel, then recombines their solutions to yield a solution that is equivalent to what would have been produced sequentially. The Parareal algorithm can be thought of as shooting-in-time due its predictor-corrector nature, or multi-grid-in-time due to its layered discretization in the time domain.

The Parareal algorithm can be broken down into four key steps

1. Prepare the subproblems
2. Solve each subproblem in parallel
3. Recombine and correct
4. Loop

The wave equation for an expanding volume is:

$$\partial_t^2\theta - \frac{\dot{a}}{a}\partial_t\theta - ac_0^2\nabla^2\theta = 0.$$

Applying a spectral decomposition in space:

$$\tilde{\theta}_{\vec{k}} = \mathcal{F}(\Delta\theta)(t,\vec{k}), \quad \text{for } k < k_c(t),$$

results in:

$$\partial_t^2\tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a}\partial_t\tilde{\theta}_{\vec{k}} - ac_0^2k^2\tilde{\theta}_{\vec{k}} = 0.$$

If the root problem P can be described by a second-order differential equation $\mathcal{L}\left(t,u,\partial_t u,\partial_t^2 u\right) = f(t)$, initial values $u(0) = u_0, \partial_t u(0) = v_0$, and time domain $D = [T, T+\Delta T]$, then define P as the initial value problem such that

$$P = \{\mathcal{L}\left(t,u,\partial_t u,\partial_t^2 u\right) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [T, T+\Delta T]\}. \tag{1}$$

Then choose a natural number N as the number of subproblems that root problem should be partitioned into; this should probably be the number of compute cores (either CPU or GPU).

Then partition the time domain for $p = 0, 1, \ldots, N-1$ so that

$$D_p = [T + p\Delta T/N,\, T + (p+1)\Delta T/N] = [T_p, T_{p+1}] \tag{2}$$

Choose a coarse propagator $\mathcal{C}_0$, e.g. semi-implicit Euler or another algorithm that is cheap to evaluate, and propagate the initial values according to the differential equation. The result of this is

$$\{u_p^0\}_p = \{u_0^0, u_1^0, u_2^0, \ldots, u_{N-1}^0\} \tag{3a}$$

$$\{v_p^0\}_p = \{v_0^0, v_1^0, v_2^0, \ldots, v_{N-1}^0\} \tag{3b}$$

where the superscript denotes that this is the zeroth-iteration. Normally, this is where the story would end, but the Parareal algorithm is just getting started.

Now the subproblem P_p^i for partition p at iteration i is defined such that

$$P_p^i = \{\mathcal{L}\left(t, u, \partial_t u, \partial_t^2 u\right) = f(t), \quad u(0) = u_p^i,\, \partial_t u(0) = v_p^i \quad D_p = [T_p, T_{p+1}]\} \tag{4}$$

The collection of subproblems at iteration i is then defined by $P^i = \{P_p^i\}_p$.

For each wave number k:

$$\partial_t^2 \tilde{\theta}_{\vec{k}} - \frac{\dot{a}}{a}\partial_t \tilde{\theta}_{\vec{k}} - a c_0^2 k^2 \tilde{\theta}_{\vec{k}} = 0, \quad \text{with initial conditions:}$$

$$\tilde{\theta}_{\vec{k}}(0) = \tilde{\theta}_{\vec{k}_0}, \quad \partial_t \tilde{\theta}_{\vec{k}}(0) = -U_0 n_{\vec{k}_0}/\hbar.$$

In addition to parallelizing, part of the magic of the Parareal algorithm lies in solving each subproblem not once, but twice with different solves or propagators. The next step in the process is to choose coarse, and fine. It should be noted that this coarse propagator does not need to be the same as the coarse propagator that was chosen in preparing the subproblems. Possible propagators include the semi-implicit Euler method with a large time step for the corase propagator, and the velocity-verlet method with a small time step for the fine propagator; in order to satisfy energy-conservation, symplectic integrators should be used. Without loss of generality, let the chosen coarse and fine propagators be denoted $\mathcal{C}, \mathcal{F}$, respectively, and the $n_{\mathcal{S}}$-th data point for propagator $\mathcal{S}$ in the p-th subprobem at iteration i be denoted $u_{pn_{\mathcal{S}}}^i$ and defined traditionally by $u_{pn_{\mathcal{S}}}^i = \mathcal{S}u_{p,\,n_{\mathcal{S}}-1}^i$, and the solution from propagator $\mathcal{S}$ on subproblem p is the ordered collection of points $\mathcal{S}u_p^i = \{u_{pn_{\mathcal{S}}}^i\}_{n_{\mathcal{S}}}$.

METHODS

PARALLEL-IN-TIME INTEGRATION AND THE PARAREAL ALGORITHM

Parallel-in-time integration attempts to solve initial value problems (IVPs) in parallel. Notable algorithms include:

- Parareal,
- Multigrid Reduction in Time (MGRIT),
- Parallel Full Approximation Scheme in Space and Time (PFASST), and others.

The Four Core Steps of the Parareal Algorithm

1. Prepare the subproblems: Given an initial value problem P:

$$\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \quad \partial_t u(0) = v_0,$$

where $D = [T, T + \Delta T]$. Choose the number of subproblems N (suggested: the number of compute cores). Partition the time domain D into subdomains D_p:

$$D_p = [T + p/N\Delta T, T + (p+1)/N\Delta T] = [T_p, T_{p+1}].$$

Use a coarse propagator $\mathcal{C}_0$ to compute initial solutions $\{u_p^0\}_p$, $\{v_p^0\}_p$.

2. Solve each subproblem in parallel: Use a coarse propagator $\mathcal{C}$ (e.g., Runge-Kutta 2 with a large time step) and a fine propagator $\mathcal{F}$ (e.g., Velocity-Verlet with a small time step). Solve each subproblem p in parallel.
3. Correct: Compute corrections for the coarse solutions using:

$$\eta_p^i = \mathcal{F}u_p^i(T_{p+1}) - \mathcal{C}u_p^i(T_{p+1}),$$

and apply corrections sequentially:

$$u_{p+1}^i(T_{p+1}) = u_p^i(T_{p+1}) + \eta_p^i.$$

4. Iterate: Repeat the process for updated initial values until convergence, e.g.:

$$|u_p^i - u_p^{i-1}| < \epsilon \quad \forall p \leq N.$$

GPU AND DISTRIBUTED COMPUTING

GPU Parallelism: GPUs, with their thousands of cores, allow solving 15,000+ subproblems concurrently, greatly enhancing accuracy compared to CPU parallelism, which typically supports only ∼10 cores.

Distributed Computing: Distributed computing frameworks like MPI enable computations across multiple machines, allowing all wave numbers to be solved simultaneously, scaling efficiently to clusters and supercomputers.

RESULTS

SOME BASIC RESULTS

Figure 1: Local CPU-based solution to $u''(t) = -u(t), u(0) = 1, u'(0) = 0$, evaluated on 24 threads.

CONCLUSION

- Equations of motion can now benefit from parallel solvers.
- Certain problems are well-suited to a divide-and-conquer approach.
- Problems with ”doubly parallel” characteristics can leverage both local and distributed parallelism, achieving significant computational efficiency.
- These advancements pave the way for modeling acoustics in expanding volumes.